5

DEBUGGING

No matter how many years of experience you have, you'll sometimes write code containing bugs. So, it's valuable to learn about the debugger and bug prevention techniques. The following questions test your ability to write code that handles errors through raising exceptions, making `assert` statements, and creating event logs with the `logging` module.

LEARNING OBJECTIVES

- Know how to make assertions with `assert` statements.
- Understand the difference between exceptions and assertions and the roles they play.
- Use the `logging` module to create a trail of clues regarding what your program is doing, and in what order.
- Be able to run your programs under the debugger to identify what is happening behind the scenes.
- Use debugger features such as breakpoints, and inspect the values stored in variables.

Practice Questions

The following questions test your ability to work with assertions, exceptions, logging, and the debugger.

Raising Exceptions

You've already practiced handling Python's exceptions with `try` and `except` statements so that your program can recover from exceptions you anticipated. But you can also raise your own exceptions. Raising an exception is a way of saying, "Stop running this code and move the program execution to the `except` statement." We raise exceptions with a `raise` statement. Answer the following questions about exceptions, the `try` and `except` statements, and `raise` statements.

1. What happens if you run the following program and press ENTER instead of entering a name?

```
print('Enter your name:')
name = input()
if name == '':
    raise Exception('You did not enter a name.')
```

```
else:
    print('Hello,', name)
```

2. Write the code that raises an `Exception` error with the error message `'An error happened. This error message is vague and unhelpful.'`

3. True or false: A `raise` statement must be inside a `try` block.

4. What happens if you run the following program and press ENTER instead of entering a name?

```
def get_name():
    print('Enter your name:')
    name = input()
    if name == '':
        raise Exception('You did not enter a name.')

    return name

try:
    name = get_name()
except:
    name = 'Guido'

print('Hello,', name)
```

Assertions

An assertion is a sanity check that makes sure your code isn't doing something obviously wrong. We make assertions with an `assert` statement. If the condition in an `assert` statement is `False`, Python raises the `AssertionError` exception. The following questions test your knowledge of `assert` statements and how to use assertions to detect problems.

5. "While exceptions are for user errors, assertions are for ____ errors."

6. Why is failing fast a good thing?

7. Which command line argument to the Python interpreter suppresses assertion checks when running a program?

8. What does `assert False` do?

Logging

Logging is a great way to understand what's happening in your program, and in what order. Python's `logging` module makes it easy to create a record of custom messages that you write.

9. Alice writes a program with several `print()` calls for debugging information instead of using the `logging` module. After she's done programming, she starts removing these `print()` calls. What are two possible mistakes she could make while removing them?

For each of the following events, decide what logging level to use for the corresponding log message. (These can be subjective and may have multiple acceptable answers.)

10. An error causes a failure that makes the program crash with no chance of recovery.

11. A particular function in your program, `calculate_my_result()`, is called.

12. The program logs the value of a particular variable.
13. The user requests that the program open a file, but the file doesn't exist.
14. The program detects that a calculation is wrong but is able to continue running.
15. The program starts running and needs to record the time and date at which it started.
16. The program keeps track of how many times a `while` loop had looped before exiting.
17. The program logs the string the user entered for an `input()` call.

Mu's Debugger

The debugger is a tool that can run a single line of code and then wait for you to tell it to continue. By running your program "under the debugger" like this, you can take as much time as you want to examine the values in the variables at any given point during the program's lifetime, making it a valuable tool for tracking down bugs. It's also more efficient than debugging your program by sprinkling `print()` calls throughout your code and rerunning it over and over.

The following questions concern the debugger for the Mu code editor used in *Automate the Boring Stuff with Python*. If you use a different debugger, try answering these questions for it instead.

18. What do you do if you want the program to run at normal speed, then pause and start the debugger once the execution reaches a particular line of code?
19. If the debugger is currently paused on a line of code within a function and you want it to run the rest of the code in the function at normal speed, then pause once the execution has returned from the function, which debugger button should you press?
20. Which button should you press if the debugger is currently paused on a line of code and you want it to resume running at normal speed?
21. If the debugger is currently paused on a line of code, how can you immediately terminate the program?
22. If the debugger is currently paused on a line of code that is a function call, which debugger button would cause the debugger to pause on the first line in that function?
23. If the debugger is currently paused on a line of code that is a function call, and you want to run all the code inside that function at normal speed, then pause again when the execution has returned from the function, which debugger button should you press?

Practice Projects

For this chapter's projects, you'll debug several programs and then write some intentionally buggy code of your own to produce different error messages.

Buggy Grade-Average Calculator

Copy the following program into your editor or download it from *https://autbor.com/buggygradeaverage.py*. This program lets the user enter any number of grades until the user enters `done`. It then displays the average of the entered grades.

```
def calculate_grade_average(grade_sum, number_of_grades):
    grade_average = int(grade_sum / number_of_grades)
    return grade_average

counter = 0
total = 0
while True:
    print('Enter a grade, or "done" if done entering grades:')
    grade = input()
    if grade == 'done':
        break
    counter = counter + 1
    total = total + int(grade)

avg = calculate_grade_average(counter, total)
print('The grade average is:', avg)
```

When you run the program and enter `100` and `50`, however, it reports the average as `0` instead of `75`:

```
Enter a grade, or "done" if done entering grades:
100
Enter a grade, or "done" if done entering grades:
50
Enter a grade, or "done" if done entering grades:
done
The grade average is: 0
```

Run this program under a debugger to find out why it doesn't work, then fix the bug. (Note that if the user enters a response other than `done` or a number, the program crashes; ignore this bug for now.)

Zero Division Error

Take a look at your corrected version of the previous grade-average program. If you run this program and immediately enter `done` without entering any grades, the program crashes with a `ZeroDivisionError: division by zero` error.

Use the debugger to find out why this happens. Add code to the `calculate_grade_average()` function so that it returns the integer `0` when the user hasn't entered any grades, instead of crashing.

Leap Year Calculator

Copy the following program into your editor or download it from *https://autbor.com/buggyLeapYear.py*. This program has an `is_leap_year()` function that takes an integer year, then returns `True` if it's a leap year and `False` if it isn't.

```
def is_leap_year(year):
    if year % 4 == 0:
        if year % 100 == 0:
            if year % 400 == 0:
                return True
            return True
        return True
    return False
```

```
while True:
    print('Enter a year or "done":')
    response = input()
    if response == 'done':
        break
    print('Is leap year:', is_leap_year(int(response)))
```

For example, if you run this program, the output will look like this:

```
Enter a year or "done":
2000
Is leap year: True
Enter a year or "done":
2001
Is leap year: False
Enter a year or "done":
2004
Is leap year: True
Enter a year or "done":
2100
Is leap year: True
Enter a year or "done":
done
```

A year is a leap year if it is evenly divisible by `4`. An exception to this rule occurs if the year is also evenly divisible by `100`, in which case it is not a leap year. There is an exception to that exception too: If the year is also evenly divisible by `400`, it is a leap year.

The year 2100 should not be a leap year, but the function call `is_leap_year(2100)` incorrectly returns `True`. Run this code under a debugger so that you can see where exactly the bug is, and then write the corrected `is_leap_year()` function.

Writing Buggy Code on Purpose

Write several short programs that produce the given error message in the following list. If you're unfamiliar with the error message, search for it on the internet to find bug reports from others who have encountered it. The filename is a hint for writing the program.

- A program named *nameError.py* that produces the error message `NameError: name 'spam' is not defined`
- A program named *badInt.py* that produces the error message `ValueError: invalid literal for int() with base 10: 'five'`
- A program named *badEquals.py* that produces the error message `SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?`
- A program named *badString.py* that produces the error message `SyntaxError: unterminated string literal (detected at line x)` (where *x* can be any number)
- A program named *badBool.py* that produces the error message `NameError: name 'true' is not defined. Did you mean: 'True'?`
- A program named *missingIfBlock.py* that produces the error message `IndentationError: expected an indented block after 'if' statement on line x` (where *x* can be any number)
- A program named *stringPlusInt.py* that produces the error message `TypeError: can only concatenate str (not "int") to str`

- A program named *intPlusString.py* that produces the error message `TypeError: unsupported operand type(s) for +: 'int' and 'str'`